Taller 3 – Diseño

Por:

Valentina Vélez Cano

Samuel Botero Gallo

Laura C. Heredia Uribe

David Anaya Negrete

Docente:

María Paula Flórez Vargas

Escuela de Ingenierías

Medellín

UPB

2026

1.1. Arquitectura de software:

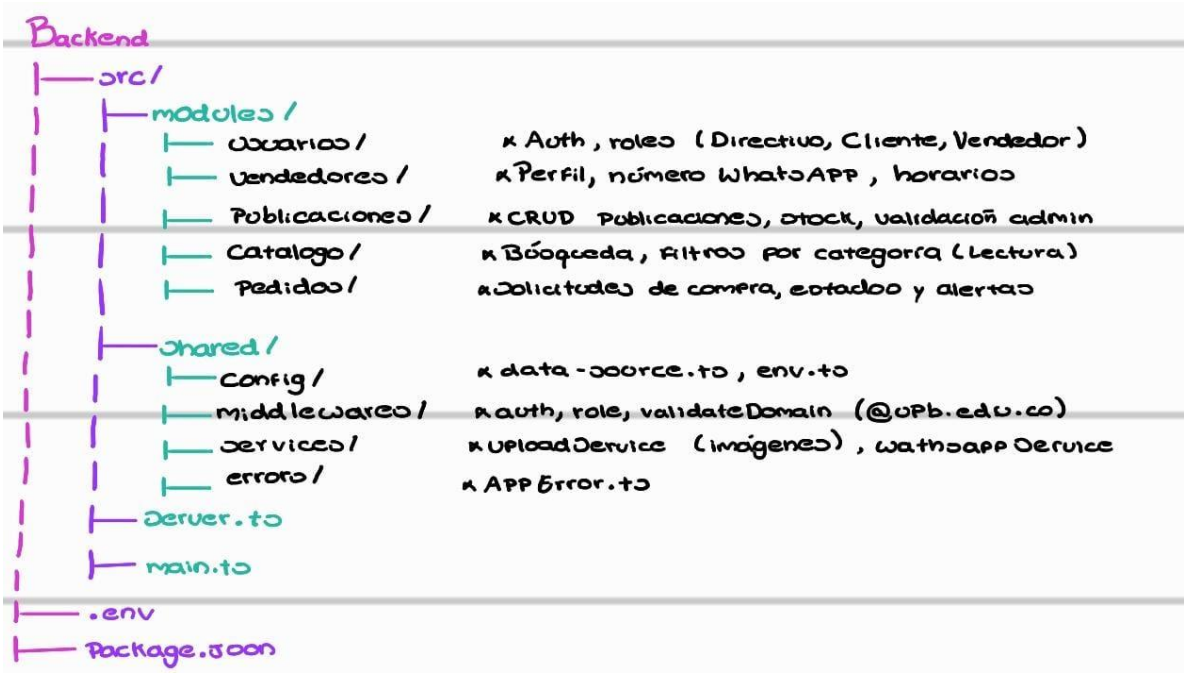
- **Estilo arquitectónico elegido:**

Debido al comportamiento del proyecto, la arquitectura que mejor se adapta es el Monolito Modular, ya que nos permite separar las distintas funciones en módulos que se enfoquen en tareas específicas, lo cual facilita modelar el comportamiento de estos de acuerdo con los distintos usuarios y sus tareas.

Otros patrones, sin embargo, no se adaptan por completo al funcionamiento del programa, ya que arquitecturas como el monolito, no permiten estructurar las distintas funcionalidades de forma tan eficiente, lo que podría dificultar llevar un orden dentro del proyecto, mientras que arquitecturas de mayor nivel como microservicios, hexagonal o clean, implican costos mayores para un proyecto que está empezando.

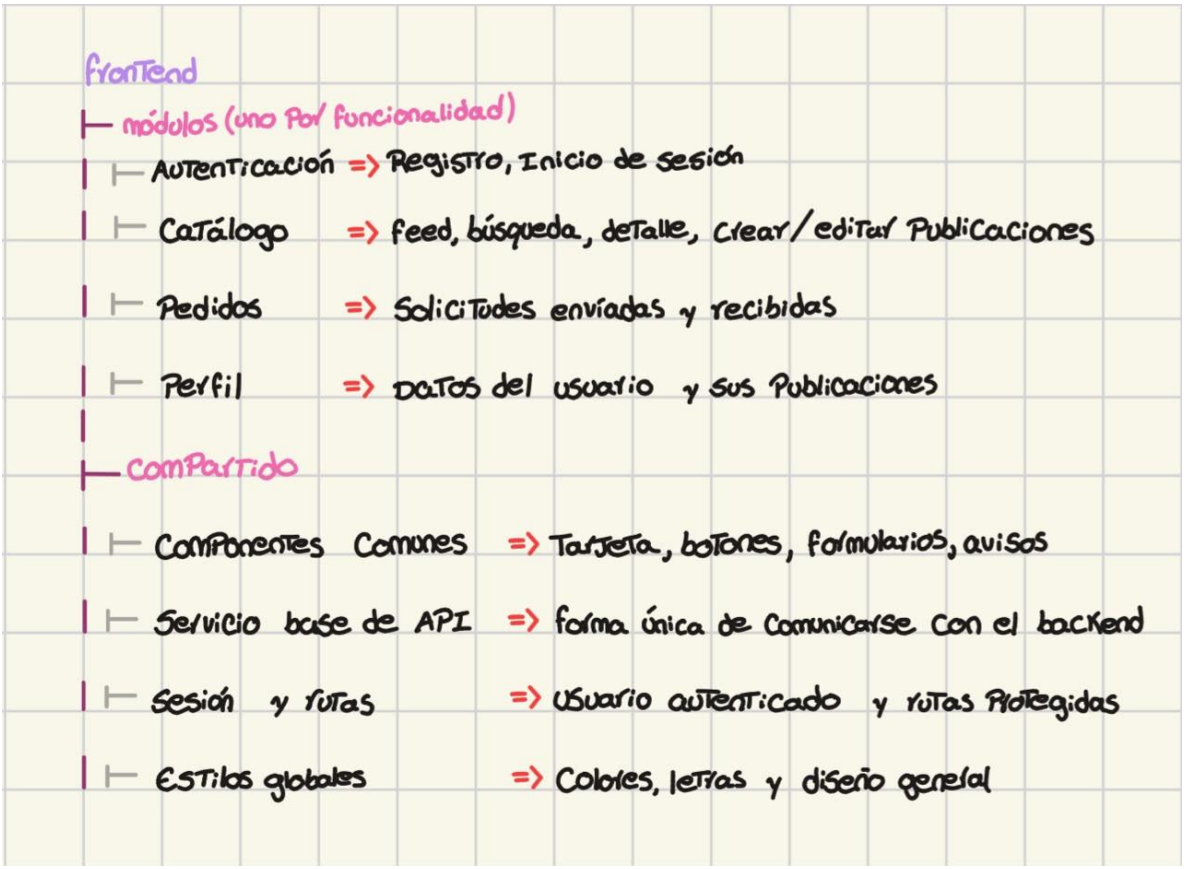
- **Estructura interna del backend:**

Para el desarrollo del backend de **Antojos UPB**, se optó por una **Arquitectura Modular basada en capas.** Esta elección permite organizar el sistema en módulos independientes alineados con las funcionalidades clave del dominio (usuarios, vendedores, publicaciones, catálogo y pedidos). Cada módulo encapsula sus responsabilidades en tres capas distintas (presentation, business y data), lo que garantiza un código altamente desacoplado, mantenible, escalable y fácil de auditar.



- **Estructura interna del frontend:**

Para el frontend decidimos organizarlo por funcionalidades, eso significa cada módulo del sistema (usuarios, catálogo, pedidos) tiene su propia parte en el frontend, y aparte hay una parte compartida con lo que usan todos. Así el equipo se puede repartir el trabajo por módulos sin escribir uno sobre el otro, y es más fácil encontrar cada cosa.



1.2. Stack tecnológico

A continuación, se definen las tecnologías seleccionadas para cada componente de la arquitectura, teniendo en cuenta los conocimientos actuales del equipo de desarrollo y los requerimientos del proyecto "Antojos UPB".

Backend (Lógica del servidor): NestJS (Node.js + TypeScript)

- Justificación: Elegimos NestJS porque su arquitectura basada en módulos encaja perfectamente con el diseño que planteamos para separar las funciones del sistema (usuarios, vendedores, catálogo, pedidos). Aunque está basado en Node.js, utiliza TypeScript de forma nativa. Esto nos beneficia enormemente porque su estructura orientada a objetos es muy similar a la lógica que ya manejamos con Java y C#, lo que nos facilita el desarrollo y mantiene el código muy organizado.

Frontend (Interfaz de usuario): React JS

- Justificación: Seleccionamos React JS porque nos permite construir la interfaz visual a través de "componentes" reutilizables. Esto se alinea con nuestra idea de tener una carpeta de "Componentes Comunes" (tarjetas, botones, formularios), agilizando el trabajo en equipo. Además, su flexibilidad nos ayudará a cumplir con el requisito de que la página sea adaptable tanto a dispositivos móviles como a computadores (responsive design).

Base de Datos: MySQL

- Justificación: Optamos por MySQL al ser un motor de base de datos relacional muy confiable y ampliamente utilizado. Esta elección es ideal porque nos permite aplicar directamente los conocimientos que estamos adquiriendo actualmente sobre SQL. El modelo relacional es perfecto para mantener la integridad de los datos y relacionar de forma segura las entidades principales de nuestro sistema: los usuarios, las publicaciones y los pedidos.

Herramientas y Servicios Adicionales:

- Gestión de Imágenes (Cloudinary): Para el módulo de publicaciones, usaremos un servicio externo en la nube como Cloudinary para almacenar las fotos de los productos. Esto evita sobrecargar nuestra base de datos relacional, haciéndola más rápida y eficiente.
- Autenticación y Seguridad (JWT + bcrypt): Para cumplir con los requisitos de seguridad, implementaremos JSON Web Tokens (JWT) y bcrypt. Esto nos permite encriptar las contraseñas de los usuarios y validar que únicamente se registren correos con el dominio institucional (@upb.edu.co).

- Despliegue (Netlify y Render): Para poner la aplicación en línea de forma gratuita durante esta etapa académica, alojaremos el Frontend en Netlify y el Backend junto con la Base de Datos en servicios en la nube como Render o Supabase.

- Contacto (API de WhatsApp): Para la comunicación entre comprador y vendedor, usaremos los enlaces directos de WhatsApp (wa.me), cumpliendo con el requisito de comunicación sin necesidad de integrar pasarelas de pago ni chats internos complejos en esta primera versión.

2. **Prototipo de Baja Fidelidad:**
   https://canva.link/blvj0qrvqfpnz2x

   Prototipo de Media Fidelidad:
   https://www.figma.com/proto/gbu90G6NZ7gUsHi2ebiBAu/Prototipo-de-Media-Fidelidad?node-id=5-6488&p=f&t=kt0uc6Ln2LoyDCdp-1&scaling=scale-down&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=5%3A6488

   Profe, este es nuestro prototipo de Media fidelidad, la mayoría de los botones funcionan, desde el inicio de sesión/crear cuenta, hasta las especificaciones de los productos, el perfil, y la visualización de crear producto nuevo y solicitudes. En la pantalla de perfil, encontrará las publicaciones activas y las publicaciones pausadas, que son productos que en el momento no cuentan con stock.